Nome do Jogo

Autores: Luis Carlos S. S. Junior, Carlos Emanuel, Nathan, Rafael Ribeiro.

Instituição: Instituto Federal do Norte De Minas Gerais - IFNMG.

Simulação Econômica / Estratégia Tycoon.

Computador (Windows).

Projeto desenvolvido para a disciplina de Economia (Curso de Sistemas de Informação).

Resumo: Um simulador *top-down* 2D onde o jogador atua como gerente de um supermercado, focado na aplicação prática de conceitos microeconômicos. O objetivo central é gerenciar o fluxo de caixa, a precificação estratégica de estoques, os custos operacionais e os gargalos de atendimento (filas dos caixas), garantindo a expansão do negócio e evitando a falência. A simulação utiliza uma inteligência artificial baseada em **utilidade marginal**, onde cada alteração de preço impacta diretamente o orçamento e a decisão de compra dos consumidores (NPCs), modelando as leis de oferta e demanda em tempo real.

Moeda Fictícia (In-Game Currency): Para fins de imersão e identidade visual, a economia do jogo não utilizará moedas do mundo real (como Real ou Dólar). O sistema financeiro será lastreado em uma moeda nativa própria (Nome a definir pela equipe), que será representada por um símbolo único na interface. Todas as variáveis de caixa (L), preço (P) e orçamento dos NPCs (B) utilizarão esta unidade base.

1. Escopo Técnico e Arquitetura Base

Engine: Godot Engine 4.7.2 (Renderização, Física, UI, Loop de Jogo, GDScript para lógicas de alto nível da interface).

Linguagem de Baixo Nível (Sistemas Críticos): C++ (via Godot GDExtension). Será utilizado para os sistemas de cálculo econômico pesado e simulação de IA de muitos NPCs, onde o desempenho é crucial.

Ambiente de Desenvolvimento:

IDE/Editor: Visual Studio Code (com integração para C++ e GDScript).

Build System: CMake (para compilar a biblioteca C++ do GDExtension) e SCons (sistema nativo de build do Godot, caso necessário como ponte).

OS Base: Linux.

Controle de Versão: Git (versionamento local) e GitHub (repositório remoto e colaboração).

2. Requisitos de Software

Vamos definir o "escopo fechado" (MVP) para garantir que o projeto não saia do controle e possa ser entregue a tempo.

2.1. Requisitos Funcionais (RF)

*(O que o jogo **tem** que fazer)*

1. **RF01 (Simulação de Precificação):** O jogador deve ser capaz de selecionar produtos (ex: Arroz, Carne, Luxo) em gôndolas e alterar seu preço de venda.
2. **RF02 (Comportamento de Compra do NPC - Demanda):** NPCs devem possuir um orçamento restrito ($) e uma lista de necessidades. A decisão de comprar um item deve considerar o preço atual vs. a utilidade do item para aquele NPC.
3. **RF03 (Gestão de Custos e Caixa):** O sistema deve cobrar taxas periódicas do jogador (aluguel, salários, reposição de estoque, água, luz).
4. **RF04 (Condição de Falência):** O jogo deve monitorar o caixa; se o saldo ficar negativo por um período de tempo determinado, o jogo deve encerrar (Game Over/Falência).
5. **RF05 (Evolução/Expansão):** O jogador deve ter a opção de investir o lucro acumulado na expansão do mercado (ex: adicionar novas gôndolas, liberar novos produtos).
6. **RF06 (Navegação):** NPCs devem ser capazes de navegar pelas gôndolas do supermercado evitando obstáculos.

2.2. Requisitos Não Funcionais (RNF)

(Como o sistema deve se comportar / restrições)

1. **RNF01 (Desempenho da IA):** O cálculo de rota (pathfinding) e decisão de compra dos NPCs deve ser feito em C++ para garantir que a simulação de 50+ NPCs simultâneos não cause *frame drops* (queda de FPS).
2. **RNF02 (Resolução):** O jogo deve suportar resoluções de tela *widescreen* padrão para PC (1920x1080), sendo desenhado inicialmente nesta proporção.
3. **RNF03 (Separação de Preocupações):** A interface do usuário (UI) não deve conter lógica complexa de economia. A interface (Godot/GDScript) deve apenas ler os dados calculados pelo backend (C++).

3. Modelagem Econômica e Regras de Negócio

Para que o jogo funcione como um simulador válido, o comportamento dos consumidores (NPCs) não pode ser aleatório. Ele deve ser guiado por regras matemáticas que simulam o conceito de **Utilidade Marginal** e **Elasticidade da Demanda**.

3.1. Estrutura de Dados do Consumidor (A Lista de Interesses)

Para otimizar o processamento da rota e simular o comportamento de consumidores reais, os NPCs não navegam aleatoriamente.

Ao ser instanciado, o sistema (C++) gera um vetor probabilístico (Array) de até 3 posições contendo categorias desejadas (ex: [Essencial, Normal, Supérfluo]).

O algoritmo de navegação (FSM) lerá este vetor linearmente. O NPC buscará a rota para a gôndola do index atual, aplicará a fórmula de Utilidade (U) daquele item específico e passará para o próximo index, até concluir o vetor e se dirigir ao caixa.

3.2. Tomada de Decisão do NPC (Demanda e Lista de Compras)

Para simular o comportamento de um consumidor real em um supermercado, os NPCs avaliarão múltiplos produtos por visita. Cada NPC gerado no jogo recebe atributos individuais:

- **Orçamento (B):** Dinheiro máximo disponível para gastar naquela visita.
- **Limiar de Exigência (T):** Quão "pão-duro" ou consumista o NPC é.
- **Lista de Interesses (L):** Um array contendo até 3 tipos de categorias de produtos que ele deseja buscar (ex: ["Essencial", "Normal", "Supérfluo"]).

O NPC navegará pelas gôndolas seguindo sua Lista de Interesses. Ao chegar na frente de um produto desejado, a Inteligência Artificial calcula a **Utilidade da Compra (U)** usando a seguinte fórmula:

$$U = \frac{N}{P} \times E$$

Onde:

1. N = Necessidade base do item (valor fixo invisível, ex: Comida = 100, Lazer = 30).
2. P = Preço de Venda atual (definido pelo jogador).
3. E = Modificador de Elasticidade (definido pelo tipo de produto).

Condição de Compra (Lógica do Carrinho):

O NPC avaliará o produto na gôndola. Ele adicionará o item ao seu "carrinho" (inventário temporário) se, e somente se, as duas condições matemáticas forem verdadeiras:

1. **Orçamento Restante:** O preço (P) é menor ou igual ao seu Orçamento (B) **atual** subtraído do valor dos itens que já estão no carrinho.
2. **Custo-Benefício:** A utilidade calculada (U) é maior ou igual ao seu Limiar (T).

Se ambas as condições forem aceitas, ele subtrai o valor de P do seu Orçamento (B) invisível, guarda o item e parte para o próximo alvo da sua Lista de Interesses. Se for rejeitado (está muito caro ou o orçamento acabou), ele ignora o produto. Após checar toda a sua lista, ele se dirige ao caixa.

3.3. Classificação e Elasticidade dos Produtos

A forma como a alteração do preço afeta a demanda varia de acordo com o tipo de produto. Produtos essenciais sofrem pouca alteração na demanda se o preço subir (demanda inelástica), enquanto supérfluos sofrem muita (demanda elástica).

Tipo de Produto	**Elasticidade (E)**	**Exemplo no Jogo**	**Comportamento do NPC**
Essencial	1.2	Arroz, Feijão, Papel Higiênico	O NPC tolera preços mais altos. A demanda cai devagar se o jogador aumentar o preço.
Normal	1.0	Biscoitos, Sucos	Comportamento padrão. Demanda cai proporcionalmente ao aumento do preço.
Supérfluo/Luxo	0.6	Carne Premium, Cerveja Premium	Altamente sensível. Um pequeno aumento no preço faz o NPC desistir da compra.

3.4. Matemática do Fluxo de Caixa do Supermercado (Oferta e Custos)

O objetivo do jogador é maximizar o Lucro (L) ao longo dos "dias" do jogo. O sistema financeiro será calculado da seguinte forma:

$$L = R_t - (C_f + C_v)$$

Onde:

- **Receita Total (R_t):** Soma de todas as compras efetivadas pelos NPCs no caixa.
- **Custo Fixo (C_f):** Despesas cobradas ao final de cada dia/mês (Aluguel, Salário dos funcionários, Água, Luz).
- **Custo Variável (C_v):** Dinheiro gasto pelo jogador para **repor o estoque** dos produtos vendidos. (Ex: o jogador vende o arroz por R$ 5,00, mas custa R$ 2,50 para repor na gôndola).

3.5. Sistema de Caixas e Gargalo de Atendimento (Filas)

Os caixas não são apenas elementos visuais; eles representam o gargalo de saída da economia do jogo.

- **Contratação e Custos:** O jogador precisará contratar NPCs Operadores de Caixa. Cada operador contratado adiciona um valor fixo mensal ao Custo Fixo (C_f) do supermercado (o salário).
- **Tempo de Atendimento:** Cada caixa tem uma velocidade de processamento (ex: demora 2 segundos reais para passar cada item do carrinho do cliente).
- **Penalidade de Fila (Desistência):** Se o jogador tiver muitos clientes (demanda alta) e poucos caixas, formar-se-á uma fila. Se o tempo de espera do NPC na fila ultrapassar um limite máximo de tolerância, o NPC **abandonará o carrinho e sairá da loja sem pagar**. Isso resulta em perda direta de Receita (R_t) e desperdício de Custo Variável (C_v) dos produtos perecíveis abandonados.
- **Decisão Estratégica:** O jogador deve equilibrar matematicamente o prejuízo de perder clientes na fila versus o custo de pagar o salário de um novo funcionário para o caixa.

3.6. Sistema de Evolução e Expansão (Upgrades)

Para que o jogo tenha um objetivo de longo prazo além da simples sobrevivência financeira, o jogador utilizará o Lucro (L) acumulado para comprar melhorias para o supermercado durante a Fase de Preparação (Mercado Fechado).

As opções de expansão incluem:

1. **Novas Gôndolas:** Permite vender novas categorias de produtos (ex: liberar a categoria "Luxo", que tem margem de lucro maior, mas atrai menos clientes).
2. **Novos Caixas (Checkouts):** Permite contratar mais funcionários para reduzir filas.
3. **Expansão de Espaço Físico (Opcional para MVP):** Aumenta o tamanho do mapa, permitindo que o sistema (Spawner) gere um limite maior de clientes simultâneos dentro da loja.

3.7. Condição de Falência

Se o caixa do jogador (Lucro acumulado) atingir um valor $L < 0$, um contador de "Dias no Vermelho" é iniciado. Se o contador chegar a 3 dias consecutivos sem que o jogador consiga

voltar a ter saldo positivo (seja vendendo estoque ou cortando gastos), a simulação é encerrada com falência.

4. Arquitetura do Sistema e Integração (Godot + C++)

O projeto utilizará o padrão arquitetural de separação de responsabilidades (MVC - Model-View-Controller adaptado para jogos). A principal justificativa para o uso do C++ não é renderizar gráficos, mas sim garantir performance na simulação de dezenas de NPCs calculando rotas e utilidade marginal simultaneamente.

4.1. Divisão de Camadas (O que faz o quê)

A estrutura do projeto será estritamente dividida em duas grandes áreas para evitar "código espaguete":

- **Frontend / View (Godot Engine & GDScript):**
  - **Responsabilidade:** Renderização 2D (Tilemaps, Sprites), Animações, Interface de Usuário (Menus, Botões, HUD de dinheiro), e detecção de cliques do mouse do jogador.
  - **Regra de Ouro:** Nenhum cálculo financeiro complexo ou lógica de decisão de IA deve ser escrito em GDScript. O Godot apenas "desenha" os dados que o C++ envia e captura as interações do jogador.
- **Backend / Model (C++ via Godot GDExtension):**
  - **Responsabilidade:** Todos os cálculos das fórmulas econômicas (U, L), controle e atualização do caixa, instanciamento dos nós de IA (decisão de compra) e cálculos pesados de atualização de posição no mapa.
  - **Regra de Ouro:** O código C++ não deve saber qual é a cor do botão na interface ou qual sprite o NPC está usando. Ele lida apenas com dados brutos (arrays, dicionários, vetores matemáticos) e lógicas de negócios.

4.2. Padrões de Projeto (Design Patterns) Adotados

Para manter o código C++ e a comunicação com o Godot organizados, a equipe utilizará os seguintes padrões de engenharia de software:

- **Singleton (Manager Pattern):**
  - Haverá uma classe global MarketManager escrita em C++. Ela guardará as variáveis globais críticas que não podem ser duplicadas (Saldo em Caixa Atual, Dia Atual, Lista de Custos Fixos). A interface (Godot) acessará esse Singleton para exibir os dados financeiros na tela.
- **Observer (Signals do Godot):**
  - Para evitar que o Godot fique verificando a cada frame (em loop) se o saldo mudou, usaremos o padrão Observer. O código em C++ emitirá um *Signal* (ex: cash_updated(new_value)) apenas no momento em que o dinheiro do jogador alterar. A UI do Godot reagirá a esse sinal atualizando o texto na tela, poupando processamento.

- **Máquina de Estados Finitos (FSM - Finite State Machine):**
    - Controlará o cérebro (IA) dos NPCs em C++. O NPC só poderá estar em um estado lógico por vez: ENTER_STORE (Entrando) -> SEARCH_ITEM (Buscando produto do Array) -> EVALUATE_PURCHASE (Calculando a fórmula de Utilidade) -> GO_TO_REGISTER (Indo ao caixa) -> LEAVE (Saindo).

4.3. Fluxo de Comunicação (Exemplo Prático)

Para ilustrar a arquitetura, este é o fluxo exato que ocorre quando o jogador altera o preço de um produto durante a jogabilidade:

1. O jogador clica no botão "Confirmar Preço: R$ 5,00" no menu do Arroz (Frontend - Godot).
2. O script do menu (GDScript) chama uma função pública exposta pelo backend: MarketManager.update_item_price("arroz", 5.00).
3. O C++ recebe a ordem, atualiza o valor no banco de dados da simulação e emite o sinal price_changed.
4. O Godot recebe o sinal e atualiza o número visualmente na etiqueta da prateleira na tela.
5. No próximo frame de simulação, todos os NPCs (C++) recalcularão a fórmula de demanda (U) usando o novo preço de R$ 5,00 caso busquem arroz.

4.4. Sistema de Navegação (Pathfinding)

Para garantir que os NPCs transitem de forma inteligente entre as gôndolas e os caixas, sem colidir com obstáculos e encontrando sempre a rota mais eficiente, o projeto implementará o algoritmo de busca *A (A-Star)**.

- **Implementação Técnica:** A equipe utilizará o NavigationServer2D nativo da Godot Engine. As prateleiras e paredes do supermercado serão mapeadas como áreas de colisão estáticas (Obstáculos), enquanto o chão gerará uma malha de navegação (NavigationPolygon).
- **Integração:** Quando a IA (C++) decidir que o NPC deve buscar um produto, ela solicitará ao servidor de navegação do Godot um *Array* de coordenadas. O NPC então se moverá interpolando sua posição através desses pontos, garantindo uma movimentação fluida e otimizada, essencial para o suporte de múltiplos clientes na tela simultaneamente.

5. Loop de Gameplay e Interface de Usuário (UX/UI)

O jogo opera em ciclos diários (turnos). O jogador não controla um avatar fisicamente; a interação é inteiramente através da Interface de Usuário (Gestão/Tycoon), influenciando a simulação que ocorre no mapa 2D abaixo dos menus.

5.1. O Ciclo Diário (Core Loop)

O ritmo do jogo é dividido em duas fases que se alternam indefinidamente, simulando o expediente comercial:

Fase 1: Preparação (Mercado Fechado)

- **Tempo:** O relógio do jogo fica pausado. Sem pressão de tempo.
- **Ações do Jogador:**
    - Verificar o saldo em caixa (Lucro/Prejuízo).
    - **Estoque:** Comprar reposição de produtos (pagando o Custo Variável - C_v) para encher as gôndolas que ficaram vazias no dia anterior.
    - **Precificação:** Ajustar o Preço de Venda (P) de cada produto, baseado no comportamento da demanda do dia anterior.
    - **Gestão (Upgrades):** Usar o lucro para comprar novas gôndolas ou contratar caixas extras.
- **Transição:** O jogador clica no botão principal da interface "Abrir Mercado".

Fase 2: Simulação (Mercado Aberto)

- **Tempo:** O relógio corre (Ex: 4 minutos reais = 1 dia de 8 horas no jogo).
- **Ações da Engine (Automáticas):**
    - NPCs começam a surgir na entrada, navegam pelas gôndolas, calculam a Utilidade (U), pegam produtos, formam filas e pagam (gerando Receita R_t).
- **Ações do Jogador (Limitadas):**
    - Durante esta fase, o jogador *apenas assiste* à simulação, coleta dados visuais (se os clientes estão achando caro ou barato) e monitora a velocidade das filas.
    - *Regra:* Não é permitido alterar preços ou comprar estoque enquanto o mercado está aberto.
- **Transição:** O relógio atinge o horário de fechamento (Ex: 22h00). As portas fecham, os NPCs restantes terminam de pagar, o sistema deduz os Custos Fixos (Aluguel, Salários, Água, Luz) e a "Tela de Relatório Financeiro" é exibida. O jogo retorna à Fase 1.

5.2. Estrutura da Interface (Godot UI)

A interface será feita no Godot, sobreposta (HUD) à visão *top-down* do supermercado.

1. **HUD Principal Superior (Sempre Visível):**
    - **Caixa:** Saldo atual (verde se > 0, vermelho se < 0, branco se = 0).
    - **Relógio:** Hora e Dia atual (Ex: "Dia 4 - 14:00").
    - **Botão de Ação Global:** "Abrir/Fechar Mercado" (Controla a mudança de Fases).
2. **Painel da Gôndola (Abre ao clicar em uma prateleira na Fase 1):**
    - Nome do Produto e Categoria (Essencial, Normal, Luxo).

    - Status do Estoque (Ex: "15/50 unidades").
    - Custo de Reposição por unidade (C_v).
    - **Controle de Preço:** Uma caixa de texto ou botões (+/-) onde o jogador define o Preço de Venda (P).
    - Botão "Comprar Reposição".
3. **Tela de Relatório Financeiro (Popup ao fim do dia):**
    - Receita Bruta (Total Vendas).
    - Gastos Fixos (C_f) e Gastos Variáveis (C_v).
    - **Lucro/Prejuízo Líquido do Dia.**
    - Contador de "Dias no Vermelho" (Aviso de falência iminente).
    - Botão "Avançar para o Próximo Dia".

5.3. Feedback Visual da Economia

Para que o jogador não precise ler planilhas para saber se a economia está funcionando, o jogo usará *feedback* visual (Sprites):

- **Emotes de Reação (Balões):** Quando um NPC avalia um produto na gôndola e rejeita a compra porque o Preço (P) está alto demais comparado à Utilidade (U), um ícone vermelho (ex: carinha brava ou cifrão cortado) aparece sobre a cabeça dele. Se ele comprar feliz, um ícone verde aparece. Isso ensina ao jogador a Elasticidade da Demanda visualmente.
- **Gôndolas Vazias:** O sprite da prateleira deve mudar conforme os itens são comprados, indicando visualmente que o estoque acabou e o jogador está perdendo dinheiro por falta de produto.
- **Filas no Caixa:** Um aglomerado de NPCs esperando para pagar é o indicador visual imediato de que o jogador precisa investir em novos funcionários de caixa.